

WEEK 5 INTERNSHIP TASK

Performance Evaluation and Optimization Strategy

Data Science Project Report

Prepared for Internship Submission

Objective: To establish a systematic framework for evaluating, monitoring, troubleshooting, and continuously optimizing a deployed data science model.

1. Introduction

A deployed data science model must be monitored continuously to ensure that it remains accurate, reliable, fast, and useful after deployment. Model performance can change because of new data, changing user behaviour, data quality problems, infrastructure limitations, or changes in the real-world environment. This report presents a practical performance evaluation and optimization strategy for a deployed machine learning solution.

The framework combines model-level metrics, system-level KPIs, monitoring, troubleshooting, and optimization into a continuous improvement cycle.

2. Task Objectives

- Measure the effectiveness of the deployed model using relevant performance metrics.
- Monitor model and system performance periodically.
- Detect data quality problems, model drift, latency, and reliability issues.
- Identify bottlenecks and possible causes of performance degradation.
- Apply optimization techniques and validate their impact.
- Create a repeatable process for continuous model improvement.

3. Performance Evaluation Framework

The evaluation framework has five major stages: data validation, model evaluation, system monitoring, issue diagnosis, and optimization. Performance should be compared against predefined thresholds and historical results.

Area	Example KPI	Purpose
Model Quality	Accuracy, Precision, Recall, F1	Measure prediction quality
Ranking/Classification	ROC-AUC	Measure class separation
Regression	MAE, MSE, RMSE	Measure prediction error
System	Latency, throughput	Measure speed and capacity
Reliability	Availability, error rate	Measure service stability
Data	Missing values, drift	Check input quality and change

4. Key Performance Metrics

4.1 Accuracy

Accuracy is the proportion of correctly predicted observations out of all observations. It is useful when classes are reasonably balanced.

Formula: $\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$

4.2 Precision

Precision measures how many of the observations predicted as positive are actually positive. It is important when false positive predictions are costly.

Formula: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

4.3 Recall

Recall measures how many actual positive observations are correctly detected. It is especially important when missing a positive case is costly.

Formula: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

4.4 F1-Score

F1-score is the harmonic mean of precision and recall and provides a balanced measure when both types of errors matter.

Formula: $\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

4.5 ROC-AUC

ROC-AUC measures how well a classification model separates positive and negative classes across different decision thresholds. A higher AUC generally indicates better discrimination.

4.6 Regression Metrics

For regression problems, Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE) can be used. Lower values indicate smaller prediction errors.

4.7 Operational Metrics

A production model should also be evaluated using response latency, requests per second, uptime/availability, failure rate, resource utilization, and prediction volume.

5. Continuous Performance Monitoring

Monitoring should run automatically and generate alerts when predefined thresholds are crossed. Model quality should be reviewed periodically, while system health can be monitored continuously.

Frequency	What to monitor	Action
Real-time	Latency, errors, availability	Trigger operational alerts
Daily	Data quality, prediction volume	Investigate unusual changes
Weekly	Model KPIs, drift indicators	Review performance trends
Monthly	Validation metrics, business KPIs	Retrain or optimize if needed

6. Monitoring Tools

- **Python and Pandas:** Data validation, metric calculation, and analysis.
- **Scikit-learn:** Classification/regression evaluation metrics and validation.
- **MLflow:** Experiment tracking, model versioning, and performance comparison.
- **Prometheus:** Collection of system and application metrics.
- **Grafana:** Dashboards and visualization of monitoring data.

- **Logging tools:** Record prediction errors, exceptions, timestamps, and service events.

7. Bottlenecks and Potential Issues

Issue	Possible Cause	Impact
Low accuracy	Poor features, outdated training data	Incorrect predictions
Model drift	Real-world data distribution changes	Performance degradation
High latency	Large model, inefficient code, overloaded server	Slow responses
Data quality problems	Missing, duplicate, or invalid values	Unreliable predictions
Class imbalance	Too few examples of minority class	Biased model
High resource use	Large model or inefficient processing	Higher infrastructure cost

8. Troubleshooting Process

- Step 1: Detect:** Identify an abnormal metric or alert.
- Step 2: Verify:** Check whether the problem is real and reproducible.
- Step 3: Diagnose:** Inspect data, logs, model outputs, and infrastructure.
- Step 4: Fix:** Apply an appropriate corrective action.
- Step 5: Validate:** Re-test the model or service using the same evaluation framework.
- Step 6: Monitor:** Observe the result over time to ensure the issue does not return.

9. Optimization Strategies

- **Hyperparameter tuning:** Use Grid Search, Random Search, or Bayesian optimization to find better model settings.
- **Feature engineering:** Create useful features, remove noisy variables, and select the most informative features.
- **Data improvement:** Handle missing values, duplicates, outliers, and inconsistent records.
- **Class imbalance handling:** Use suitable sampling methods, class weights, or threshold tuning.
- **Model selection:** Compare alternative algorithms using consistent validation data.
- **Cross-validation:** Evaluate generalization more reliably across multiple data splits.
- **Model efficiency:** Reduce model size, simplify preprocessing, cache repeated operations, or use efficient inference.
- **Retraining:** Retrain when drift or sustained performance decline is detected.

10. Monitoring and Optimization Flow

1. Collect Data	→	2. Validate Data	→	3. Run Model
↑		↓		



The process is iterative: performance is measured, problems are identified, improvements are applied, and the updated model is evaluated again before being promoted to production.

11. Example Performance Dashboard

Metric	Target	Status / Interpretation
F1-Score	>= 0.85	Healthy if above target
Precision	>= 0.85	Controls false positives
Recall	>= 0.85	Controls missed positives
Latency	< 200 ms	Suitable for responsive service
Availability	>= 99%	Reliable service
Data Missing Rate	< 2%	Good input quality

12. Expected Improvements

After optimization, the project should achieve more stable model quality, lower prediction errors, faster response time, better data quality, fewer production failures, and improved resource efficiency. Every optimization should be validated against a baseline so that improvements are measurable rather than assumed.

13. Conclusion

Performance evaluation does not end when a machine learning model is deployed. Continuous monitoring is necessary to ensure that the model remains accurate and operationally reliable. By tracking model KPIs, system metrics, data quality, and drift indicators, potential problems can be detected early. A structured troubleshooting and optimization process allows the team to retrain, tune, or improve the system when performance falls below acceptable levels.

The proposed framework provides a practical cycle for maintaining a deployed data science solution: measure → monitor → diagnose → optimize → validate → repeat.

14. References

- Scikit-learn documentation – Model evaluation and metrics.
- MLflow documentation – Experiment tracking and model lifecycle management.
- Prometheus documentation – Monitoring and alerting.
- Grafana documentation – Metrics visualization and dashboards.